

Advanced Java & Spring Boot

Complete Master Study Notes & Architectural Reference Guide

JAVA 17+ RECORDS & SEALED

SPRING BOOT 3

SPRING DATA JPA & MONGO

SPRING SECURITY 6 & JWT

FINITE STATE MACHINE (FSM)

JUNIT 5 TESTING

Advanced Java & Spring Boot – Complete Master Study Notes & Reference Guide

Repository: [Advance-Java-Practice](#)

Author: Siam

Topic: Comprehensive Notes on Advanced Java, Spring Boot, JPA/Hibernate, MongoDB, Security (JWT & Form), Cloud Services, Testing & Enterprise Patterns

Table of Contents

1. Learning Journey & Architectural Roadmap
2. Modern Java (Java 17+) Language Features
3. Java Records & Compact Constructors
4. Sealed Classes & Permitted Subclasses
5. Lombok Productivity Suite
6. Spring Boot Core & Web MVC Framework
7. Core Stereotype Annotations & Inversion of Control (IoC)
8. Dependency Injection Best Practices
9. RESTful Controllers vs Traditional MVC Controllers

10. Request Parameter Handling
11. Form Submission, Thymeleaf & Model Binding
12. Relational Persistence: Spring Data JPA & Hibernate
13. Entity Mapping & Identifiers
14. Embedded Objects vs Element Collections
15. Entity Cardinality & Relational Mappings
16. Cascade Types & Fetch Strategies
17. Repository Abstraction, Derived Queries & Custom JPQL
18. Declarative Transactions (@Transactional)
19. NoSQL Persistence: Spring Data MongoDB
20. Document Mapping & Indexing
21. Auditing (@EnableMongoAuditing, @CreatedDate, @LastModifiedDate)
22. Optimistic Locking with @Version
23. Custom JSON Queries (\$or, \$ne, regex)
24. Enterprise REST API Design, DTOs & Error Handling
25. The DTO (Data Transfer Object) Pattern
26. Input Validation (Jakarta Bean Validation)
27. Centralized Exception Handling (@RestControllerAdvice)
28. Cross-Origin Resource Sharing (CORS) Configuration
29. Cloud Integrations: Media Management with Cloudinary
30. Multipart File Uploads
31. Cloudinary Service Implementation
32. Microservices, External REST APIs & Caching
33. Consuming External REST APIs with RestTemplate
34. Spring Cache Abstraction (@EnableCaching, @Cacheable, @CacheEvict)
35. Spring Security 6: Form-Based & Stateless JWT
36. Security Architecture Overview
37. Password Encoding with BCrypt
38. Form-Based Security with Custom AuthenticationProvider
39. Stateless JWT Architecture (Filter, Claims, Token Lifecycle)
40. Method-Level Security (@EnableMethodSecurity, @PreAuthorize)
41. Advanced Enterprise Design Patterns
 - Finite State Machine (FSM) Domain Workflow
 - Pagination & Sorting (Pageable & Page)
 - Automatic Master Data Seeding (CommandLineRunner)

42. Unit & Integration Testing (JUnit 5 & Spring Boot Test)

- Test Lifecycle Annotations
- Assertions & Verification
- Testing Spring Data Services

43. Workspace Project Inventory & Reference Map

1. Learning Journey & Architectural Roadmap

The code in this repository traces an end-to-end evolution from core language primitives to full-scale enterprise software architecture:

[Modern Java: Records & Sealed Classes]

|

▼

[Spring Boot MVC & Web Controllers] → [Thymeleaf + Form Validation]

|

▼

▼ [Relational Database: JPA / MySQL]

- Entity Mappings & @Id
- Embedded Objects & @ElementCollection
- Cardinality (@OneToOne, @ManyToOne, @OneToMany)
- JPQL Queries & @Transactional

▼ [NoSQL Database: MongoDB]

- @Document & @Indexed
- Auditing & Optimistic Locking
- Spring Data MongoDB
- Custom JSON Queries

|

▼

[Enterprise REST APIs & Error Handling]

- DTO Pattern & Bean Validation (@Valid)
- Global Exception Handler (@RestControllerAdvice)
- Cloudinary Media / Image Uploads

|

▼

[Microservices, External APIs & Caching]

- RestTemplate Consumption of Third-Party REST APIs
- Spring Cache Abstraction (@Cacheable, @CacheEvict)

|

▼

[Spring Security 6 Architecture]

- Form-Based + Custom AuthenticationProvider
- Stateless JWT Authentication (Filter Chain, Tokens)
- Role-Based Access Control (RBAC) & Method Security

|

▼

[Advanced Domain Architecture]

- Finite State Machine (Clinical FSM Workflow)
- Pagination & Sorting (Pageable, Page<T>)
- Database Seeding & Optimistic Concurrency Control

2. Modern Java (Java 17+) Language Features

Found in project: **Record**

Java Records & Compact Constructors

Records were introduced to eliminate boilerplate when defining immutable data carriers: – Automatically provides **private final** fields, canonical constructor, getters (**name()**, **gpa()**), **equals()**, **hashCode()**, and **toString()**. – Supports **Compact Constructors** for normalization and validation without repeating parameter assignments.

```
package org.example.DTO;  
  
public record StudentGpaRecord(  
    String name,  
    double gpa  
) {  
    public StudentGpaRecord {  
        if (name == null || name.isBlank()) {  
            throw new IllegalArgumentException("Student name cannot be empty");  
        }  
        name = name.toUpperCase();  
        if (gpa > 4.0) {  
            gpa = 4.0;  
        }  
    }  
}
```

Sealed Classes & Permitted Subclasses

Sealed classes allow superclasses or interfaces to strictly control which classes can extend or implement them, enabling safe pattern matching and domain modeling.

```
package org.example;

public sealed class Car permits ElectricCar, WaterCar {
    private int id;
    private String brand;
    private String color;
}

public final class ElectricCar extends Car {
    private double batteryCapacityKWh;
}

public final class WaterCar extends Car {
    private double propellerSize;
}
```

Lombok Productivity Suite

Lombok reduces boilerplate across entities and DTOs at compile-time:

Annotation	Purpose
<code>@Data</code>	Generates <code>@Getter</code> , <code>@Setter</code> , <code>@ToString</code> , <code>@EqualsAndHashCode</code> , and <code>@RequiredArgsConstructor</code> .
<code>@NoArgsConstructor</code>	Generates a zero-argument constructor (required by Hibernate/JPA and MongoDB reflection).
<code>@AllArgsConstructor</code>	Generates a constructor initializing all fields.
<code>@Builder</code>	Implements the Builder Pattern for readable, fluent object creation.
<code>@Builder.Default</code>	Ensures default field values are preserved when using the builder.
<code>@RequiredArgsConstructor</code>	Generates a constructor for all <code>final</code> fields (standard for Spring Dependency Injection).
<code>@Slf4j</code>	Injects an SLF4J logger instance (<code>log.info(...)</code> , <code>log.error(...)</code>).

3. Spring Boot Core & Web MVC Framework

Found in projects: `Hello Spring Boot`, `courierDemo`, `Hello Rest`

Core Stereotype Annotations & Inversion of Control (IoC)

Spring manages beans in the ApplicationContext: - `@SpringBootApplication`: Meta-annotation combining `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`. - `@Component`: Generic Spring-managed component. - `@Service`: Marks business logic layer. - `@Repository`: Marks persistence layer (enables automatic exception translation into Spring `DataAccessException`). - `@Controller`: Handles Web/HTML views (returns view names to Thymeleaf/JSP). - `@RestController`: Combines `@Controller` and `@ResponseBody` (returns serialized JSON/XML). - `@Configuration` + `@Bean`: Declares explicit bean definitions (e.g., custom security filters, external clients).

Dependency Injection Best Practices

Prefer **Constructor Injection** using Lombok `@RequiredArgsConstructor` over field injection (`@Autowired`): -
 Immutable dependencies (`private final`). - Prevents `NullPointerException` during standalone unit testing. - Guarantees objects are fully initialized upon instantiation.

```
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;
}
```

RESTful Controllers vs Traditional MVC Controllers

```
@Controller
public class WebController {
    @GetMapping("/dashboard")
    public String dashboard(Model model) {
        model.addAttribute("username", "Siam");
        return "dashboard";
    }
}

@RestController
@RequestMapping("/api/v1/items")
public class ItemRestController {
    @GetMapping
    public List<Item> getItems() {
        return itemService.getAll();
    }
}
```

Request Parameter Handling

- `@PathVariable`: Extracts values from URI template paths: `/api/cases/{id}` -> `public Case getById(@PathVariable String id)`
- `@RequestParam`: Extracts query parameters: `/api/cases?status=OPEN` -> `public List<Case> getByStatus(@RequestParam String status)`

- `@RequestBody` : Deserializes incoming JSON request body into a Java object: `public Case create(@RequestBody CaseDTO dto)`
- `@ModelAttribute` : Binds form parameters from HTML POST requests to a model object.

Form Submission, Thymeleaf & Model Binding

Found in `courierDemo/CourierController.java` :

```
@Controller
public class CourierController {
    private final CourierRepository courierRepository;

    @GetMapping("/book")
    public String showBookForm(Model model) {
        model.addAttribute("courier", new Courier());
        return "book";
    }

    @PostMapping("/book")
    public String processBooking(
        @Valid @ModelAttribute("courier") Courier courier,
        BindingResult bindingResult,
        Model model) {

        if (bindingResult.hasErrors()) {
            return "book";
        }
        courierRepository.save(courier);
        return "redirect:/history";
    }
}
```

4. Relational Persistence: Spring Data JPA & Hibernate

Found in projects: `Product-Shop (1)-(4)` , `Hotel-Management-System` , `Security-Project`

Entity Mapping & Identifiers

```
@Entity
@Table(name = "rooms")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Room {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "room_number", nullable = false, unique = true)
    private String roomNumber;

    @Enumerated(EnumType.STRING)
    @Column(name = "room_type", nullable = false)
    private RoomType type;

    @Column(name = "price_per_night", nullable = false)
    private Double pricePerNight;
}
```

Embedded Objects vs Element Collections

Found in `Product-Shop (4) (DatabaseCardinality)/Student.java` : - `@Embeddable` & `@Embedded` : Merges properties of a value object directly into the host entity table (no separate table or foreign key). -

`@ElementCollection` : Creates a joined secondary table for collections of basic types (Strings, Integers) or embeddables without an independent entity identity.

```

@Embeddable
public class Address {
    @Column(name = "street_address")
    private String streetAddress;
    private String city;
    private String state;
    private String country;
}

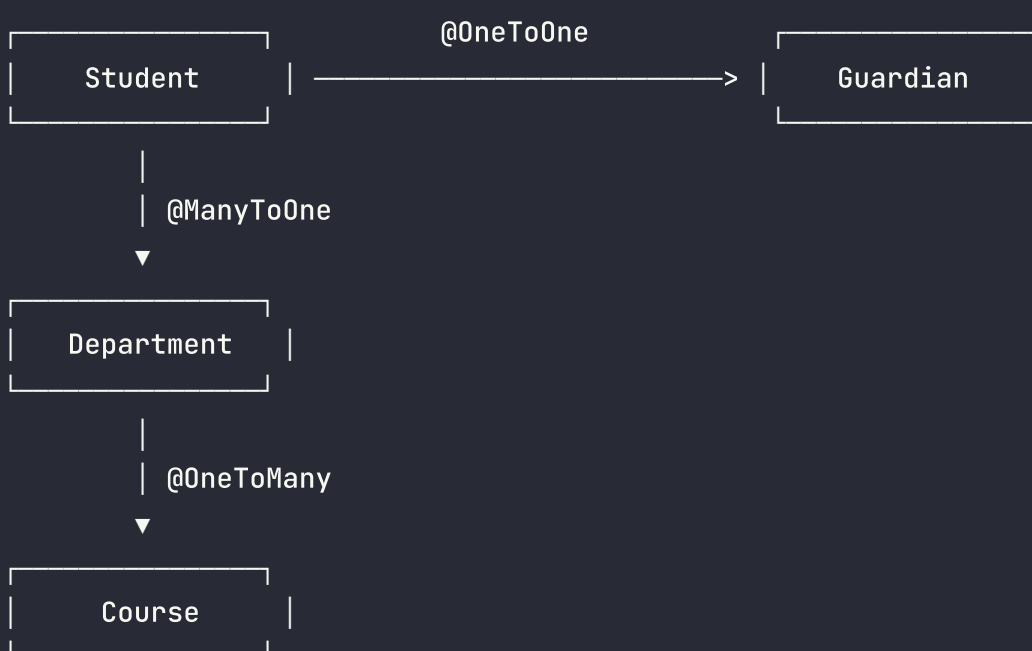
@Entity
@Table(name = "student_data")
public class Student {
    @Id
    private int id;
    private String name;

    @Embedded
    private Address address;

    @ElementCollection
    private List<String> mobileNumber;
}

```

Entity Cardinality & Relational Mappings



1. **One-to-One** (`@OneToOne`): `java @OneToOne private Guardian guardian;`
2. **Many-to-One** (`@ManyToOne`): `java @ManyToOne(fetch = FetchType.EAGER) @JoinColumn(name = "guest_id", nullable = false) private Guest guest;`
3. **One-to-Many** (`@OneToMany`): `java @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY) private List<Course> courses;`

Cascade Types & Fetch Strategies

- **CascadeType:**
 - `CascadeType.ALL` : Propagates PERSIST, MERGE, REMOVE, REFRESH, DETACH from parent to children.
 - `CascadeType.PERSIST` : Only saves children when parent is saved.
- **FetchType:**
 - `FetchType.LAZY` : Child data loaded on-demand when getter is called (prevents N+1 query performance bottleneck).
 - `FetchType.EAGER` : Joined/queried immediately when loading the parent.

Repository Abstraction, Derived Queries & Custom JPQL

Found in `Hotel-Management-System/BookingRepository.java` :

```
@Repository
public interface BookingRepository extends JpaRepository<Booking, Long> {
    List<Booking> findByStatus(BookingStatus status);
    List<Booking> findByGuestId(Long guestId);
    List<Booking> findTop5ByOrderByIdDesc();

    @Query("SELECT SUM(b.totalAmount) FROM Booking b WHERE b.status ≠ 'CANCELLED'")
    Double calculateTotalRevenue();
}
```

Declarative Transactions (@Transactional)

Found in `Hotel-Management-System/BookingService.java` : - `@Transactional` ensures atomicity: if any step fails with an unchecked exception (`RuntimeException`), all database changes in the method rollback. - Manages Hibernate persistence context and dirty checking (modifications to attached entities are automatically saved upon transaction commit).

```
@Transactional
public void checkIn(Long bookingId) {
    Booking booking = bookingRepository.findById(bookingId)
        .orElseThrow(() -> new IllegalArgumentException("Booking not found: " + bookingId));
    booking.setStatus(BookingStatus.CHECKED_IN);

    Room room = booking.getRoom();
    room.setStatus(RoomStatus.OCCUPIED);
}
```

5. NoSQL Persistence: Spring Data MongoDB

Found in projects: [Mastery-Case-Tracker](#), [Lifecycle Management](#), [Lost-Found](#), [siam](#)

Document Mapping & Indexing

```
@Document(collection = "cases")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CaseRecord {
    @Id
    private String id;

    @Indexed(unique = true)
    @Field("case_id")
    private String caseId;

    @Field("title")
    private String title;

    @Field("priority")
    private Priority priority;

    @Field("status")
    private CaseStatus status;

    @CreatedDate
    @Field("created_at")
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Field("updated_at")
    private LocalDateTime updatedAt;
}
```

Auditing (@EnableMongoAuditing, @CreatedDate, @LastModifiedDate)

By placing `@EnableMongoAuditing` on a `@Configuration` class (`MongoConfig.java`), Spring automatically sets `@CreatedDate` when saving new records and `@LastModifiedDate` when updating existing records.

Optimistic Locking with @Version

Found in `Lifecycle Management/Patient.java` : - Prevents the **Lost Update Problem** in concurrent environments. - Spring Data tracks a version number. If two users read version `1` and attempt to update

concurrently, the second write fails with `OptimisticLockingFailureException`.

```
@Document(collection = "patients")
public class Patient {
    @Id
    private String id;

    @Version
    private Long version;

    private String name;
    private ClinicalStatus clinicalStatus;
}
```

Custom JSON Queries (\$or, \$ne, regex)

Found in `Lifecycle Management/PatientRepository.java`:

```
@Repository
public interface PatientRepository extends MongoRepository<Patient, String> {
    @Query("{ 'clinicalStatus' : { $ne: ?0 } }")
    List<Patient> findActivePatients(ClinicalStatus excludedStatus);

    @Query("{ $or: [ " +
        " { 'name' : { $regex: ?0, $options: 'i' } }, " +
        " { 'mrn' : { $regex: ?0, $options: 'i' } }, " +
        " { 'department' : { $regex: ?0, $options: 'i' } } " +
        "]" })
    List<Patient> searchPatients(String regex);
}
```

6. Enterprise REST API Design, DTOs & Error Handling

Found in projects: `Mastery-Case-Tracker`, `Lifecycle Management`

The DTO (Data Transfer Object) Pattern

Separating internal persistence entities (`@Entity` , `@Document`) from external API contracts: - **Security:** Prevents over-posting/mass-assignment attacks. - **Decoupling:** Database schema changes don't break external API clients. - **Performance:** Avoids sending redundant or sensitive internal data (passwords, internal IDs).

```

Client Request → [CaseRequestDTO] → [Service Layer] → [CaseRecord (Entity)] →
Client Response ← [CaseResponseDTO] ←

```

Input Validation (Jakarta Bean Validation)

Applied to incoming DTOs in controllers using `@Valid @RequestBody` :

```

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CaseRequestDTO {
    @NotBlank(message = "Case ID is required")
    @Pattern(regexp = "^CASE-\\d{3,}$", message = "Case ID must follow format CASE-")
    private String caseId;

    @NotBlank(message = "Case Title is required")
    @Size(min = 3, max = 150, message = "Case Title must be between 3 and 150 characters")
    private String title;

    @NotNull(message = "Priority is required (HIGH, MEDIUM, LOW)")
    private Priority priority;
}

```

Centralized Exception Handling (@RestControllerAdvice)

Found in `Mastery-Case-Tracker/GlobalExceptionHandler.java` : - Intercepts exceptions thrown anywhere in the controller layer. - Returns standardized `ErrorResponse` objects with proper HTTP status codes.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        ErrorResponse error = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.NOT_FOUND.value())
            .error("Not Found")
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .build();
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationErrors(MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(err ->
            fieldErrors.put(err.getField(), err.getDefaultMessage())
        );

        ErrorResponse error = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Validation Failed")
            .message("Input validation errors occurred")
            .path(request.getRequestURI())
            .validationErrors(fieldErrors)
            .build();
        return ResponseEntity.badRequest().body(error);
    }
}

```

Cross-Origin Resource Sharing (CORS) Configuration

To allow frontend SPAs (React, Angular, Vue) to interact with the backend API:

```
@Configuration
public class WebCorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("```
```

```
---
```

```
## 7. Cloud Integrations: Media Management with Cloudinary
```

```
Found in project: `Lost-Found`
```

```
### Multipart File Uploads
```

```
For uploading images and files alongside form data:
```

```
```java
@PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<Item> createItem(
 @ModelAttribute ItemRequestDto dto,
 @RequestParam(value = "image", required = false) MultipartFile image) throws
 Item created = itemService.createItem(dto, image);
 return ResponseEntity.status(HttpStatus.CREATED).body(created);
}
```

## Cloudinary Service Implementation

Configures Cloudinary credentials from `application.properties` and handles upload and deletion:

```
@Configuration
public class CloudinaryConfig {
 @Bean
 public Cloudinary cloudinary(
 @Value("${cloudinary.cloud-name}") String cloudName,
 @Value("${cloudinary.api-key}") String apiKey,
 @Value("${cloudinary.api-secret}") String apiSecret) {
 return new Cloudinary(ObjectUtils.asMap(
 "cloud_name", cloudName,
 "api_key", apiKey,
 "api_secret", apiSecret));
 }
}

@Service
@RequiredArgsConstructor
public class CloudinaryService {
 private final Cloudinary cloudinary;

 public Map<String, Object> uploadImage(MultipartFile file) throws IOException {
 return cloudinary.uploader().upload(file.getBytes(), ObjectUtils.asMap(
 "folder", "lost-found-items",
 "resource_type", "auto"
));
 }

 public void deleteImage(String publicId) {
 try {
 cloudinary.uploader().destroy(publicId, ObjectUtils.emptyMap());
 } catch (IOException e) {
 log.error("Failed to delete Cloudinary asset: {}", publicId, e);
 }
 }
}
```

## 8. Microservices, External REST APIs & Caching

Found in project: **Hello Microservice**

## Consuming External REST APIs with RestTemplate

Using Spring's `RestTemplate` to make HTTP calls and map JSON responses to domain models:

```
@Service
public class UserService {
 private final RestTemplate restTemplate = new RestTemplate();

 public List<User> getAllUsers() {
 String url = "https://jsonplaceholder.typicode.com/users";
 User[] users = restTemplate.getForObject(url, User[].class);
 return Arrays.asList(users);
 }

 public User getUserById(int id) {
 String url = "https://jsonplaceholder.typicode.com/users/" + id;
 return restTemplate.getForObject(url, User.class);
 }
}
```

## Spring Cache Abstraction (@EnableCaching, @Cacheable, @CacheEvict)

Eliminates redundant external API calls or expensive database lookups by storing results in-memory:

1. **Enable Caching:** Add `@EnableCaching` to main application class or configuration.
2. `@Cacheable`: Checks cache first. If found, returns cached value without executing the method.
3. `@CacheEvict`: Invalidates entries when data is modified to prevent stale data.

```

@Service
public class PatientServiceImpl implements PatientService {

 @Override
 @Cacheable(value = "patients", key = "#id")
 public PatientResponse getPatientById(String id) {
 return patientRepository.findById(id)
 .map(this::toResponse)
 .orElseThrow(() -> new PatientNotFoundException(id));
 }

 @Override
 @CacheEvict(value = {"patients", "dashboard"}, allEntries = true)
 public PatientResponse updatePatient(String id, PatientRequest request) {
 ...
 }
}

```

## 9. Spring Security 6: Form-Based & Stateless JWT

Found in projects: [Hello Spring Security](#), [Security-Project](#), [Lifecycle Management](#)

### Security Architecture Overview

Spring Security operates as a chain of servlet filters ([SecurityFilterChain](#)):

HTTP Request —> [CorsFilter] —> [CsrfFilter] —> [JwtAuthenticationFilter] —> [U

### Password Encoding with BCrypt

Never store plain-text passwords. BCrypt incorporates a random salt and adaptive work factor:

```

@Bean
public PasswordEncoder passwordEncoder() {
 return new BCryptPasswordEncoder();
}

```

## Form-Based Security with Custom AuthenticationProvider

Found in `Hello Spring Security`: Implements `AuthenticationProvider` to support custom user authentication and role lookup:

```
@Component
@RequiredArgsConstructor
public class CustomAuthenticationProvider implements AuthenticationProvider {
 private final UserService userService;
 private final PasswordEncoder passwordEncoder;

 @Override
 public Authentication authenticate(Authentication authentication) throws AuthenticationException {
 String email = authentication.getName();
 String password = authentication.getCredentials().toString();

 User user = userService.findByEmail(email);
 if (user == null || !passwordEncoder.matches(password, user.getPassword()))
 throw new BadCredentialsException("Invalid email or password");
 }

 List<GrantedAuthority> authorities = user.getRoles().stream()
 .map(SimpleGrantedAuthority::new)
 .collect(Collectors.toList());

 return new UsernamePasswordAuthenticationToken(email, null, authorities);
}

@Override
public boolean supports(Class<?> authentication) {
 return UsernamePasswordAuthenticationToken.class.isAssignableFrom(authentication);
}
}
```

## Stateless JWT Architecture (Filter, Claims, Token Lifecycle)

Found in `Lifecycle Management`: For modern stateless REST APIs where the server stores no session (`SessionCreationPolicy.STATELESS`):



## 1. JwtAuthenticationFilter:

```

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {
 private final JwtUtils jwtUtils;
 private final UserDetailsService userDetailsService;

 @Override
 protected void doFilterInternal(
 @NonNull HttpServletRequest request,
 @NonNull HttpServletResponse response,
 @NonNull FilterChain filterChain) throws ServletException, IOException {

 final String authHeader = request.getHeader("Authorization");
 if (authHeader == null || !authHeader.startsWith("Bearer ")) {
 filterChain.doFilter(request, response);
 return;
 }

 final String token = authHeader.substring(7);
 final String username = jwtUtils.extractUsername(token);

 if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
 UserDetails userDetails = userDetailsService.loadUserByUsername(username);
 if (jwtUtils.isTokenValid(token, userDetails)) {
 UsernamePasswordAuthenticationToken authToken =
 new UsernamePasswordAuthenticationToken(userDetails, null,
 authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request)));
 SecurityContextHolder.getContext().setAuthentication(authToken);
 }
 }
 filterChain.doFilter(request, response);
 }
}

```

## 2. Stateless SecurityFilterChain Configuration:

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
 private final JwtAuthenticationFilter jwtAuthFilter;

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
 http
 .csrf(AbstractHttpConfigurer::disable)
 .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
 .authorizeHttpRequests(auth -> auth
 .requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()
 .requestMatchers(HttpMethod.DELETE, "/api/patients").hasRole("ADMIN")
);

 ### Method-Level Security (@EnableMethodSecurity, @PreAuthorize)
 Allows fine-grained security directly on controller or service methods:

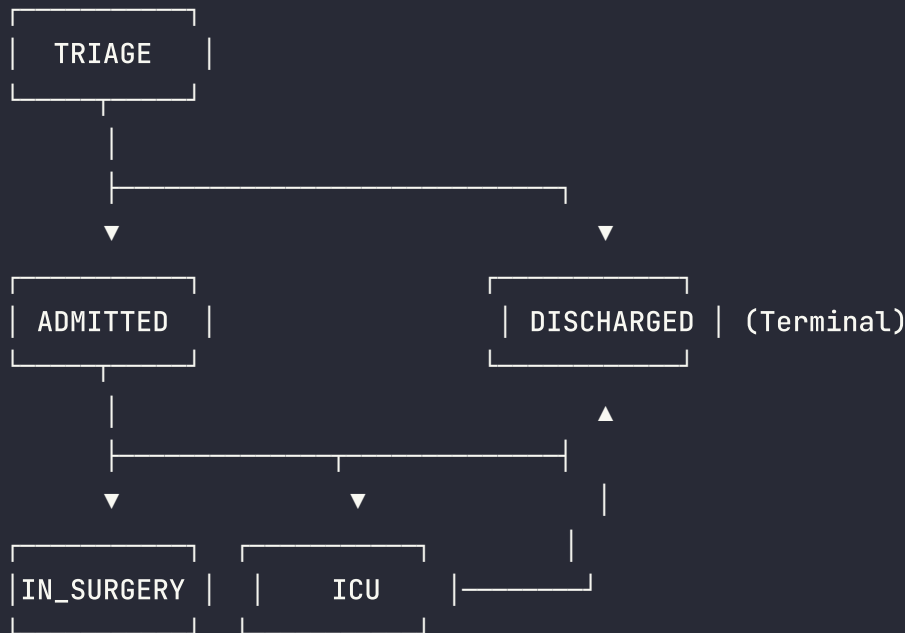
        ```java
        @PreAuthorize("hasRole('ADMIN')")
        @DeleteMapping("/{id}")
        public ResponseEntity<Void> deletePatient(@PathVariable String id) {
            patientService.deletePatient(id);
            return ResponseEntity.noContent().build();
        }
    
```

10. Advanced Enterprise Design Patterns

Found in project: **Lifecycle Management**

Finite State Machine (FSM) Domain Workflow

The hospital patient management system implements a Finite State Machine guarding clinical transitions:



```

@Service
public class ClinicalFsmService {
    private static final Map<ClinicalStatus, Set<ClinicalStatus>> TRANSITIONS = new HashMap<>();

    static {
        TRANSITIONS.put(ClinicalStatus.TRIAGE, Set.of(ClinicalStatus.ADMITTED, ClinicalStatus.DISCHARGED));
        TRANSITIONS.put(ClinicalStatus.ADMITTED, Set.of(ClinicalStatus.IN_SURGERY, ClinicalStatus.ICU));
        TRANSITIONS.put(ClinicalStatus.IN_SURGERY, Set.of(ClinicalStatus.ICU, ClinicalStatus.DISCHARGED));
        TRANSITIONS.put(ClinicalStatus.ICU, Set.of(ClinicalStatus.DISCHARGED));
        TRANSITIONS.put(ClinicalStatus.DISCHARGED, Set.of());
    }

    public void validateTransition(ClinicalStatus current, ClinicalStatus next) {
        if (current == next) return;
        Set<ClinicalStatus> allowed = TRANSITIONS.getOrDefault(current, Set.of());
        if (!allowed.contains(next)) {
            throw new IllegalClinicalStateTransitionException(current.name(), next.name());
        }
    }
}

```

Pagination & Sorting (Pageable & Page)

Crucial for handling large datasets without loading hundreds of thousands of records into memory:

```

@GetMapping
public ResponseEntity<Page<PatientResponse>> getAllPatients(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "createdAt") String sortBy,
    @RequestParam(defaultValue = "desc") String direction) {

    Sort sort = direction.equalsIgnoreCase("asc") ? Sort.by(sortBy).ascending() : Sort.by(sortBy).descending();
    Pageable pageable = PageRequest.of(page, size, sort);
    return ResponseEntity.ok(patientService.getAllPatients(pageable));
}

```

Automatic Master Data Seeding (CommandLineRunner)

Automatically creates default administrative accounts and baseline data when the application boots:

```

@Component
@RequiredArgsConstructor
public class DataSeeder implements CommandLineRunner {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Override
    public void run(String... args) {
        if (userRepository.count() == 0) {
            ApplicationUser admin = ApplicationUser.builder()
                .username("admin")
                .password(passwordEncoder.encode("admin123"))
                .roles(Set.of("ROLE_ADMIN", "ROLE_PHYSICIAN"))
                .build();
            userRepository.save(admin);
        }
    }
}

```

11. Unit & Integration Testing (JUnit 5 & Spring Boot Test)

Found in projects: [UnitTest](#), [Hello Mongo Test](#), [Mastery-Case-Tracker](#)

Test Lifecycle Annotations

Found in `UnitTest/MathServiceTest.java` :

Annotation	Execution Timing	Note
<code>@BeforeAll</code>	Once before all tests in the class	Must be <code>static</code> method
<code>@BeforeEach</code>	Before every individual <code>@Test</code>	Initializes fresh test state
<code>@Test</code>	The actual test case	Marked methods must not return a value
<code>@AfterEach</code>	After every individual <code>@Test</code>	Cleanups / state reset
<code>@AfterAll</code>	Once after all tests complete	Must be <code>static</code> method

```
@SpringBootTest
public class MathServiceTest {

    @Autowired
    private MathService mathService;

    @BeforeAll
    public static void beforeAllTest() {
        System.out.println("Executing class-level setup...");
    }

    @Test
    void addTest() {
        int result = mathService.add(1, 2);
        Assertions.assertEquals(3, result, "1 + 2 should equal 3");
    }

    @Test
    void sub() {
        int result = mathService.sub(5, 1);
        Assertions.assertEquals(4, result);
    }
}
```

Testing Spring Data Services

Found in `Hello Mongo Test/StudentServiceTest.java` :

```
@SpringBootTest
public class StudentServiceTest {
    @Autowired
    private StudentService studentService;

    @Test
    public void testSaveStudent() {
        StudentSaveDTO dto = new StudentSaveDTO("Mr. Java", 48, 3.52, "approved");
        Student saved = studentService.save(dto);

        Assertions.assertNotNull(saved.getId());
        Assertions.assertEquals(dto.name(), saved.getName());
    }
}
```

12. Workspace Project Inventory & Reference Map

Project Name	Primary Technology	Key Architectural Features Demonstrated
<code>Record</code>	Java 17 Core	Records, Compact Constructors, Sealed Classes (<code>permits</code> , <code>final</code>), DTOs.
<code>Hello Spring Boot (1)</code>	Spring MVC	<code>@Controller</code> , <code>@GetMapping</code> , <code>@PostMapping</code> , Request parameter binding, Web forms.
<code>courierDemo</code>	Spring MVC + JPA	Thymeleaf forms, Jakarta validation (<code>@NotBlank</code> , <code>@Positive</code>), <code>BindingResult</code> , PRG pattern.
<code>Product-Shop (1-4)</code>	Spring Data JPA + MySQL	JPA Entities, <code>@Embedded</code> , <code>@ElementCollection</code> , Cardinalities (<code>@OneToOne</code> , <code>@ManyToOne</code> , <code>@OneToMany</code>), Cascade & Fetch types.
<code>Hotel-Management-System</code>	Full-Stack Spring Data JPA	Multi-entity relational system (Rooms, Guests, Bookings), Custom JPQL queries, <code>@Transactional</code> services, Enums (<code>EnumType.STRING</code>).
<code>siam & Lifecycle Manager</code>	Spring Data Mongo (Early)	MongoDB document storage, manual statistics aggregation, CRUD operations.
<code>Hello Rest</code>	Spring Data Mongo	Clean REST Controller structure (<code>@RestController</code> , <code>@RequestMapping</code> , <code>@RequestBody</code> , <code>@PathVariable</code>).
<code>Mastery-Case-Tracker</code>	Spring Data Mongo + REST	Full DTO architecture, Regex validation, Auditing (<code>@EnableMongoAuditing</code>), <code>@RestControllerAdvice</code> Global Exception Handling, CORS.
<code>Lost-Found</code>	Spring Boot + Cloudinary	Multipart file uploads (<code>MultipartFile</code>), Cloudinary SDK integration (upload, delete), PATCH endpoints.

Project Name	Primary Technology	Key Architectural Features Demonstrated
Hello Microservice	Spring Boot + RestTemplate	Consuming third-party external REST APIs, JSON deserialization, Spring Cache (@EnableCaching , @Cacheable).
Hello Spring Security	Spring Security 6 (Form)	Form login, custom AuthenticationProvider , BCrypt password hashing, session management, role/authority authorization.
Security- Project	Spring Security 6	Web route authorization, SecurityFilterChain configuration, Thymeleaf security tag library.
Lifecycle Management	Enterprise Full-Stack	Stateless JWT Authentication, Custom UserDetailsService, Method Security (@PreAuthorize), Finite State Machine (Clinical FSM), Spring Caching (@Cacheable , @CacheEvict), Optimistic Locking (@Version), Pagination (Pageable), Data Seeding.
UnitTest & Hello Mongo Test	JUnit 5 + Spring Boot Test	Test lifecycle callbacks (@BeforeAll , @BeforeEach , etc.), Assertions, Testing Spring services with real databases.

Summary Checklist for Coding & Interviews

- Model Immutability:** Use Java

record

 for DTOs and value carriers; use

@Entity

 or

@Document

 for persistent models.
- Layer Separation:** Controller (API routing) -> Service (Business logic & transactions) -> Repository (Data access).
- Database Relationships:** Always specify

fetch

 strategy (

LAZY

 for collections) and

@JoinColumn

 on owning sides.
- Validation:** Never trust client input; use

@Valid

 with Jakarta annotations (

@NotBlank

 ,

@Pattern

 ,

@Size

) and handle failures with

@RestControllerAdvice

 .
- Security:** Use stateless JWT for APIs; hash passwords with

BCryptPasswordEncoder

 ; protect routes with filter chain rules and

@PreAuthorize

 .

6. **State Machines:** Never mutate complex state blindly; encapsulate state transition rules inside a dedicated FSM service.
 7. **Concurrency:** Always guard critical concurrent documents with `@Version` optimistic locking.
 8. **Performance:** Utilize Spring Caching (`@Cacheable` , `@CacheEvict`) and database pagination (`Pageable` , `Page<T>`) for scalable systems.
-

Advanced Java Practice • Generated from Study Notes • Author: Siam